Architectural Decisions

ChatGPT Reserach Link Reference: https://chatgpt.com/share/6a0de007-c314-83e9-b6cb-bc0de03ead30

Risks and Trade-offs

1. Purpose

This document describes the main architectural risks, trade-offs, and mitigations for the blog platform technical exercise.

The goal is to show that the solution was designed intentionally, not only from a technical preference perspective, but also considering the challenge constraints, implementation time, maintainability, and presentation clarity.

Documenting these risks is useful for the interview because it demonstrates awareness of practical software engineering concerns such as scope control, security, testing, architecture boundaries, and responsible use of tooling.

---

2. Summary of Selected Architecture

The project is a small full-stack blog platform built with a .NET backend, React frontend, PostgreSQL database, and a Clean Architecture-inspired structure.

The selected architecture is:

- Backend runtime: .NET 10 LTS
- API style: ASP.NET Core Web API with controllers
- Frontend: React + Vite + TypeScript
- Database: PostgreSQL
- Data access: Raw SQL with Npgsql and custom repositories/gateways
- Authentication: JWT Bearer authentication
- Authorization: ASP.NET Core built-in authorization with roles/policies where appropriate
- Password handling: Secure password hashing

- API documentation: OpenAPI documentation, with Swagger UI only if useful for local demonstration
- Error handling: ProblemDetails-style responses
- Testing: xUnit, unit tests, API integration tests, and PostgreSQL-backed data access tests where practical
- Architecture style: Domain, Application, Infrastructure/Data, API, Tests, and React frontend layers

This architecture was chosen because it is modern, stable, explainable, and aligned with the technical challenge. It avoids Entity Framework, Dapper, Mediator, and unnecessary enterprise patterns while still demonstrating clear separation of concerns, authentication, authorization, CRUD operations, custom data access, testing, and frontend integration.

---

3. Key Risks, Trade-offs, and Mitigations

Area	**Risk / Trade-off**	**Impact**	**Mitigation**	**Interview Explanation**
Scope	Overengineering the solution	The project could become too large to finish, test, or explain clearly.	Keep the blog platform small. Focus on posts, tags, categories, users, roles, and reactions only. Avoid CMS features such as comments, drafts, rich publishing workflows, notifications, or analytics.	The goal is not to build a full CMS. The goal is to demonstrate architecture, authentication, authorization, CRUD, testing, and frontend integration in a focused project.
Runtime	Choosing an unstable or risky SDK/runtime version	Preview or unsupported versions could create setup issues during review.	Use .NET 10 LTS and avoid preview-only APIs or experimental features. Document the SDK version in the README.	.NET 10 LTS provides a modern but stable baseline. The project prioritizes reliability and presentation safety over using the newest possible feature.

Data Access	Custom data access complexity without Entity Framework or Dapper	Raw SQL requires more manual mapping, more boilerplate, and more testing.	Keep repositories small, use clear SQL scripts, isolate SQL in Infrastructure, and avoid generic repository abstractions.	Raw SQL is intentionally used to satisfy the challenge and demonstrate direct understanding of persistence, transactions, and query behavior.
Security	SQL injection risk when using raw SQL	Unsafe string concatenation could expose the database to injection attacks.	Use parameterized SQL only. Never concatenate user input into SQL commands. Review repository methods carefully.	Raw SQL is safe when parameterized correctly. The implementation keeps all SQL access isolated and testable.
Authentication	Authentication or JWT implementation mistakes	Weak token validation, incorrect claims, or poor expiration handling could make the API insecure.	Use ASP.NET Core JWT Bearer authentication. Keep token generation isolated in a service. Configure issuer, audience, signing key, and expiration consistently.	JWT is appropriate for a React + Web API demo, but security-sensitive code is kept small, explicit, and reviewed carefully.
Authorization	Authorization bugs around post ownership and admin-only endpoints	Users could edit/delete posts they do not own, or access administrator features.	Add ownership checks in application services. Use `[Authorize]` for protected endpoints and role/policy checks for admin actions. Add tests for forbidden scenarios.	Authorization is handled at two levels: endpoint access through ASP.NET Core and business rules through application services.
Testing	Weak or superficial test coverage	The project may appear to have tests without proving the	Prioritize tests for ownership rules, admin-only category	Tests focus on the highest-risk behavior rather than only testing

		important business rules.	management, login failures, validation errors, and repository behavior.	simple happy paths.
Data Tests	Data access tests becoming too slow or hard to maintain	Repository tests may slow development or become fragile if database setup is complex.	Use a dedicated PostgreSQL test database. Keep seed/reset scripts simple. Test only critical repository behavior.	Data access tests are limited but meaningful. They verify the custom SQL layer without turning the test suite into a maintenance burden.
Frontend	Frontend scope growing too large	The React app could consume too much time and distract from backend requirements.	Limit frontend pages to the demo flow: public posts, post details, login/register, my posts, post form, and admin categories. Avoid Redux or complex UI frameworks.	The frontend demonstrates integration and usability, but the main evaluation focus remains the full-stack architecture and backend rules.
API Errors	Poor error response consistency	The frontend and reviewers may see inconsistent error formats.	Use ProblemDetails-style responses for validation, not found, unauthorized, forbidden, conflict, and unexpected errors.	Consistent error responses make the API easier to consume, test, and explain.
Documentation	OpenAPI/Swagger documentation becoming outdated	API documentation could stop matching implementation.	Generate OpenAPI from the ASP.NET Core API. Keep DTOs and response types explicit. Use Swagger UI only as a local demo convenience if needed.	API documentation is generated from the implementation to reduce manual drift.

GenAI Usage	GenAI-generated code introducing hidden mistakes	Generated code may violate constraints, introduce security issues, or add unnecessary complexity.	Manually review all generated code. Validate against official documentation and project constraints. Add tests for generated logic.	GenAI is used as an assistant, not as an authority. Final decisions and code are reviewed and validated manually.
Seed Data	Seeded data or demo credentials being handled carelessly	Demo credentials could be confused with production practices.	Use clearly documented demo-only users. Do not use real secrets. Keep local secrets out of source control.	Seeded credentials exist only to make the technical review easy to run and demonstrate.
Local Setup	Docker/local setup becoming too complex	Reviewers may have trouble running the project.	Keep Docker Compose focused on PostgreSQL and optionally the API/frontend if practical. Provide clear README commands.	The local setup should reduce friction, not become another large part of the challenge.
Architecture	Clean Architecture becoming too abstract for a small project	Too many abstractions could make the project harder to understand.	Apply Clean Architecture lightly. Use clear layers, but avoid unnecessary patterns such as CQRS, MediatR, generic repositories, or excessive interfaces.	The architecture is used to protect boundaries and testability, not to demonstrate unnecessary complexity.

4. Accepted Trade-offs

Raw SQL instead of an ORM

Raw SQL is more verbose than using an ORM, but it satisfies the challenge requirement to avoid Entity Framework and Dapper. This trade-off is accepted because it demonstrates understanding of database access,

SQL queries, parameterization, transactions, and repository design.

The mitigation is to keep SQL isolated in Infrastructure repository classes and expose only clean repository interfaces to the Application layer.

Controllers instead of Minimal APIs

Controller-based APIs are more explicit and slightly more verbose than Minimal APIs. This trade-off is accepted because controllers make the project easier to review and present during an interview.

Controllers provide a familiar structure for routes, request DTOs, response DTOs, authorization attributes, validation behavior, and OpenAPI documentation.

JWT authentication instead of a full identity platform

JWT authentication requires careful implementation, but it is appropriate for a small React + Web API project. A full identity platform would be more complete, but it could hide important implementation details and add unnecessary complexity.

The project keeps authentication simple: registration, login, password hashing, JWT creation, and ASP.NET Core token validation.

Simple frontend instead of a complex frontend architecture

The frontend is intentionally small. It should demonstrate the required user flows, not become a large application.

This trade-off keeps the focus on full-stack integration, authentication, authorization, CRUD operations, and business rules.

Lightweight Clean Architecture instead of enterprise architecture

Clean Architecture is applied to create clear boundaries between Domain, Application, Infrastructure, API, and Frontend. However, the project avoids excessive abstractions.

This trade-off keeps the codebase understandable while still demonstrating maintainable design.

---

5. Risk Mitigation Plan

- Keep the domain small and focused on the required blog platform behavior.
- Implement the happy path first: register, login, list posts, create post, edit own post, delete own post.

- Add authorization checks early instead of treating them as a final step.
- Write tests for post ownership rules.
- Write tests for administrator-only category management.
- Write tests for public versus protected endpoints.
- Use parameterized SQL only.
- Keep all SQL isolated in Infrastructure repository classes.
- Avoid leaking Npgsql, SQL commands, or table structure into the Application layer.
- Use a real PostgreSQL test database for critical repository tests where practical.
- Keep database setup scripts simple and repeatable.
- Document seeded users and demo credentials clearly in the README.
- Keep frontend pages limited to the required demo flow.
- Avoid Redux or complex frontend state management unless absolutely necessary.
- Use ProblemDetails-style responses consistently.
- Keep OpenAPI documentation generated from the API implementation.
- Review all GenAI-generated code manually.
- Validate important architectural and security decisions against official documentation.
- Avoid adding libraries unless they clearly reduce risk or improve clarity.
- Keep Docker Compose focused on local development needs.
- Document known limitations honestly.

---

6. Final Position

The selected architecture intentionally balances modern .NET practices, technical challenge constraints, implementation time, and interview clarity.

The solution uses stable and explainable technologies: .NET 10 LTS, ASP.NET Core controllers, PostgreSQL, raw SQL with Npgsql, JWT authentication, ProblemDetails-style error responses, xUnit tests, OpenAPI documentation, and a simple React + Vite frontend.

The main risk is not technical feasibility, but scope control. For that reason, the project should remain a focused blog platform rather than a full CMS. The architecture is designed to demonstrate clean boundaries, security awareness, custom data access, testability, and full-stack integration without unnecessary complexity.